



دانشگاه شریف

بسته‌ی پروژه‌ی پایان‌ترم دانشجو
درس ساختار کامپیوتر و میکروپروسسور

شرح کامل پروژه، مجموعه‌دستور، برنامه‌ی زمانی و شیوه‌ی نمره‌دهی

طراحی و پیاده‌سازی پردازنده‌ی PARS-16 با نرم‌افزار Digital

ددلاین : ۲۰ مرداد ۱۴۰۵

۱۰ مرداد ۱۴۰۵

فهرست مطالب

۱	مقدمه، هدف و قوانین کلی پروژه	۱
۲	۱-۱ این پروژه چیست	۲
۲	۲-۱ اهداف یادگیری	۲
۳	۳-۱ پیش نیازها	۳
۳	۴-۱ نصب و راه اندازی ابزار	۳
۴	۵-۱ صداقت علمی و قوانین تحویل	۴
۴	۱-۵-۱ قالب تحویل	۴
۵	۲ مرجع کامل مجموعه دستور PARS-16	۵
۵	۱-۲ مشخصات کلی (الزامی و ثابت برای همه ی گروه ها)	۵
۶	۲-۲ رجیسترها	۶
۷	۳-۲ مدل حافظه	۷
۷	۴-۲ رجیستر فلگ و اجرای شرطی سراسری	۷
۸	۱-۴-۲ رجیستر فلگ	۸
۹	۲-۴-۲ بیت S: فعال سازی به روزرسانی فلگ	۹
۹	۳-۴-۲ کد شرط دوبیتی (CC)	۹
۱۰	۴-۴-۲ مقایسه چگونه انجام می شود	۱۰

۱۰	۵-۴-۲ الگوی پرش شرطی: چرا دو سیکل کلاک
۱۰	۶-۴-۲ شبیه‌سازی شرط‌های معکوس (GE، NE، ...)
۱۱	۵-۲ فرمت‌های دستور
۱۵	۶-۲ جدول کامل دستورات
۱۶	۷-۲ شبه‌دستورها
۱۷	۱-۷-۲ محدودیت LI و ساختن ثابت‌های بزرگ
۱۸	۸-۲ نحو زبان اسمبلی
۱۸	۱-۸-۲ مثال کامل: جمع اعداد 1 تا N
۲۰	۳ راهنمای معماری میکرو و قرارداد رابط اجباری
۲۰	۱-۳ مسیر داده‌ی تک‌سیکل
۲۰	۱-۱-۳ نمای کلی بلوکی
۲۱	۲-۱-۳ ترتیب پیشنهادی ساخت اجزا
۲۲	۲-۳ جدول سیگنال‌های کنترلی
۲۳	۱-۲-۳ پیشنهاد برای ساخت خودِ واحد کنترل
۲۳	۳-۳ قرارداد پین الزام سخت برای نمره‌دهی خودکار
	۱-۳-۳ نکته‌ی پیاده‌سازی: پورت خواندنِ سوم رجیستر فایل رجیستر فایل: دو
۲۷	پورت خواندن، نه سه
۲۸	۴-۳ سؤالات متداول طراحی
۲۹	مراجع

فصل ۱

مقدمه، هدف و قوانین کلی پروژه

بازنگری ۶ مرداد - این صفحه را حتماً بخوانید

این نسخه چند تغییر نسبت به نسخه‌ی ۲۴ تیر دارد. همه‌ی تغییرات در متن با خط‌خورده (حذف‌شده) و هایلایت (جایگزین) مشخص شده‌اند، پس لازم نیست کل سند را دوباره بخوانید. خلاصه:

۱. الزام پورت خواندن سوم رجیستر فایل حذف شد. دو پورت کافی است؛ پنجره‌ی دیباگ با یک MUX روی آدرس پورت دوم و پین جدید DBG_EN ساخته می‌شود (بخش ۳-۳-۱).

۲. دستور HALT و پین HALT_OUT حذف شدند. کد عملیات 1111 فقط NOP است و پایان برنامه یک پرش به خود است. این کار مسیر داده را ساده‌تر می‌کند.

۳. دو پین جدید برای مشاهده‌ی حافظه‌ی داده اضافه شد: DBG_MADDR و DBG_MOUT (بخش ۳-۳).

۴. محدودیت LI برای ساختن ثابت‌های بزرگ صریح شد (بخش ۲-۷-۱).

۵. آدرس حافظه‌ی داده 16 بیت شد (قبلاً 8 بیت نوشته شده بود). این مستقل از PC است و باعث می‌شود خروجی ALU بدون هیچ Splitter میانی مستقیم به RAM برود.

۶. رفتار دستورهای پرش با PC هشت‌بیتی صریح تعریف شد: مقصد پرش پیمانه‌ی 256، برداشتِ هشت بیت پایین Rs1 در JR، و گسترش با صفرِ PC+1 در JAL (بخش ۲-۶).

۷. تنظیم اجباری قطعات حافظه اضافه شد — برچسب DMEM/IMEM و تیک Program Memory. این بخش در نسخه‌ی قبلی جا افتاده بود و بدون آن هیچ تستی روی مدار شما اجرا نمی‌شود (بخش ۳-۳).

۸. یک مثال کامل و آزموده‌شده‌ی رجیستر فایل به همراه فایل تست و راهنمای مستقل، در پوشه‌ی examples/regfile اضافه شد. پیشنهاد می‌کنیم کار را از همان‌جا شروع کنید.

عرض PC و آدرس حافظه‌ی دستور بدون تغییر و همان 8 بیت است؛ آدرس حافظه‌ی داده 16 بیت شد (مورد ۵).

۱-۱ این پروژه چیست

در این پروژه، در بازه‌ی زمانی سه هفته، یک پردازنده‌ی کامل و کارکننده به نام PARS-16 را طراحی و شبیه‌سازی خواهید کرد. PARS-16 یک مجموعه‌دستور اختصاصی این درس است: نه RISC-V واقعی است و نه MIPS واقعی، اما دقیقاً با همان روحیه‌ی RISC طراحی شده دستورهای با طول ثابت، فرمت‌های منظم و تعداد کم دستور و یک ویژگی خاص و آموزنده دارد که در بخش ۲-۴ توضیح داده می‌شود: اجرای شرطی سراسری^۱ روی تمام دستورات، نه فقط پرش‌ها.

۲-۱ اهداف یادگیری

- درک این‌که یک مجموعه‌دستور^۲ چگونه به مسیر داده و سیگنال‌های کنترلی نگاشت می‌شود.
- تجربه‌ی عملی ساخت رجیستر فایل، ALU، واحد کنترل و حافظه از قطعات پایه.
- فهم عمیق‌تر مفهوم تک‌سیکل از طریق ساختن (نه فقط خواندن).

^۱ Predicated Execution

^۲ ISA

- آشنایی با یک روش واقعی متفاوت برای شرطگذاری در دستورها (اجرای پیش بینی شده، الگوی معماری ARM) و مقایسه‌ی آن با الگوی رایج‌تر «پرش شرطی» در RISC-V/MIPS.
- تجربه‌ی نوشتن برنامه‌ی اسمبلی برای سخت‌افزاری که خودتان ساخته‌اید.

۳-۱ پیش نیازها

فرض می‌شود مباحث زیر را از پیش می‌دانید: جبر بول و دروازه‌های منطقی، مالتی‌پلکسر و دیکودر، فلیپ‌فلاپ و رجیستر، حافظه‌ی ROM/RAM، و مفهوم کلاک/سیگنال همگام. اگر در هرکدام مطمئن نیستید، پیش از شروع هفته‌ی اول مرور کنید.

۴-۱ نصب و راه‌اندازی ابزار

۱. Java Runtime Environment (نسخه‌ی ۸ یا بالاتر) را نصب کنید.
۲. آخرین نسخه‌ی Digital را از مخزن رسمی آن دریافت کنید: <https://github.com/hneemann/> digital (بخش Releases).
۳. فایل Digital.jar را اجرا کنید. برای اجرای دستورهای خط‌فرمانی که در این سند می‌بینید، باید بدانید Digital.jar در کجای سیستم شما قرار دارد و از همان مسیر در پنجره‌ی فرمان (Command Prompt یا PowerShell) اجرا شود.
۴. پیشنهاد: پیش از شروع، پوشه‌ی examples همراه نرم‌افزار (از طریق منوی باز کردن نمونه‌ها در برنامه) را باز کنید و پردازنده‌ی نمونه‌ی تک‌سیکل شبیه MIPS که در آن پیدا می‌کنید را مطالعه کنید. این مثال برای الهام گرفتن از سبک طراحی بسیار مفید است (چطور رجیستر فایل ساخته شده، چطور کنترلر کار می‌کند)، اما مجموعه‌دستور آن با PARS-16 یکی نیست و طبق قانون زیر کپی مستقیم آن ممنوع است.

۵-۱ صداقت علمی و قوانین تحویل

ممنوعیت صریح

- import کردن یا کپی کردن مستقیم مدار پردازنده‌ی نمونه‌ی داخلی Digital (یا هر پردازنده‌ی آماده‌ی دیگر پیدا شده در اینترنت) ممنوع است. مجموعه‌دستور PARS-16 به‌اندازه‌ی کافی متفاوت است که چنین کاری عملاً کار نمی‌کند، اما تلاش برای آن نیز خودش تخلف محسوب می‌شود.
- اشتراک‌گذاری فایل dig. ممنوع است.
- دستیاران آموزشی از دستور CLI stats برای انگشت‌نگاری ساختاری مدارهای تحویلی استفاده می‌کنند؛ موارد مشابه برای بررسی دستی پرچم‌گذاری می‌شوند.

۱-۵-۱ قالب تحویل

هر فرد دقیقاً یک فایل NN_CPU.dig (به‌جای NN شماره‌ی دانشجویی) به‌همراه هر زیرمدار کمکی که با import در آن استفاده کرده (همه در یک پوشه‌ی فشرده) تحویل می‌دهد. اگر طراحی سلسله‌مراتبی (چند فایل dig) دارید، تمام فایل‌ها را در یک پوشه‌ی فشرده‌ی واحد با نام NN.zip تحویل دهید تا وابستگی‌های import سالم بمانند (به‌یاد داشته باشید Digital فایل‌های import شده را صرفاً بر اساس نام‌شان نه مسیرشان پیدا می‌کند؛ پس همه‌چیز را کنار هم نگه دارید).

الزام حیاتی: پین‌های ورودی/خروجی فایل بالادستی (یعنی همان NN_CPU.dig) باید دقیقاً با «قرارداد پین» فصل ۳-۳ مطابقت داشته باشند در غیر این صورت هیچ‌کدام از تست‌های خودکار روی مدار شما اجرا نمی‌شوند.

فصل ۲

مرجع کامل مجموعه دستور PARS-16

۱-۲ مشخصات کلی (الزامی و ثابت برای همه‌ی گروه‌ها)

مشخصه	مقدار
عرض کلمه (داده و دستور)	16 bit
تعداد رجیستر عمومی	8 عدد (R7—R0)
معماری حافظه	Harvard — حافظه‌ی دستور و داده کاملاً مجزا
آدرس‌دهی	Word-addressed (هر آدرس = یک کلمه‌ی 16 بیتی، نه بایت)
اندازه‌ی حافظه‌ی دستور (ROM)	دقیقاً 256 کلمه — آدرس 8 bit
اندازه‌ی حافظه‌ی داده (RAM)	دقیقاً 256 کلمه — آدرس 8 bit آدرس 16 bit
تعداد کد عملیات	16 دستور (تمام فضای opcode پر می‌شود)
مسیر داده	تک‌سیکل ^۱ (اجباری برای نمره‌ی کامل)

چرا اندازه‌ی حافظه ثابت است

برخلاف بسیاری از تمرین‌های معماری که اندازه‌ی حافظه را «به انتخاب شما» می‌گذارند، اینجا اندازه‌ی حافظه‌ی دستور (و در نتیجه عرض PC) **عمداً ثابت و اجباری** است: تست خودکار بر پایه‌ی پین PC_OUT با عرض دقیق 8 bit کار می‌کند. اگر عرض آن را تغییر دهید، تست‌های خودکار روی مدار شما اجرا نخواهند شد.

تصحیح: آدرس حافظه‌ی داده 16 بیت است، نه 8

عرض PC و عرض آدرس حافظه‌ی داده دو چیز مستقل‌اند و نباید با هم اشتباه شوند. معماری Harvard است، پس دو مسیر آدرس جدا وجود دارد:

- آدرس حافظه‌ی دستور از PC می‌آید $\rightarrow$ 8 بیت (بدون تغییر).
- آدرس حافظه‌ی داده از خروجی ALU می‌آید $(Rs1 + sext(imm4))$ و چون رجیسترها 16 بیتی‌اند، این آدرس ذاتاً 16 بیت است.

اگر آدرس RAM را 8 بیت بگذاریم، مجبورید وسط مسیر داده یک Splitter بگذارید تا هشت بیت پایین خروجی ALU را جدا کنید. این کار نه آموزشی است و نه بی‌خطر: بیت‌های بالا بی‌صدا دور ریخته می‌شوند و آدرس‌های بزرگ‌تر به‌جای خطا، بی‌سروصدا دور می‌زنند. با 16 بیت، خروجی ALU مستقیم به RAM می‌رود و این مرحله کلاً حذف می‌شود.

هزینه‌اش صفر است: RAM با 16 بیت آدرس در Digital کاملاً عادی است و شما در عمل فقط چند ده خانه‌ی اولش را استفاده می‌کنید.

۲-۲ رجیسترها

هشت رجیستر عمومی 16 بیتی با قرارداد زیر:

رجیستر	نام مستعار	کاربرد قراردادی
R0	ZERO	همیشه مقدار 0؛ نوشتن روی آن بی اثر است (مثل RISC-V/MIPS)
R6—R1	—	رجیسترهای عمومی، آزاد
R7	RA	رجیستر بازگشت ^۱ ؛ توسط JAL به طور خودکار نوشته می شود

رجیستر فایل شما باید علاوه بر دو پورت خواندن معمولی داخل مسیر داده (برای Rs1 و Rs2)، یک پورت خواندن سوم فقط برای دیباگ هم داشته باشد (بخش ۳-۳) که هیچ اثری روی اجرای برنامه ندارد و صرفاً برای تست خودکار استفاده می شود.

۲-۳ مدل حافظه

معماری Harvard: حافظه‌ی دستور (ROM) و حافظه‌ی داده (RAM) کاملاً جدا هستند و آدرس یکسان در این دو به معنای دو مکان متفاوت است. این ساده‌سازی از دردسرهای معماری von Neumann (تداخل واکشی دستور و دسترسی داده در یک حافظه‌ی مشترک) در مسیر داده‌ی تک‌سیکل جلوگیری می‌کند — دقیقاً همان انتخابی که نسخه‌ی آموزشی تک‌سیکل MIPS در کتاب پترسون-هنسی هم می‌کند [۱].

هیچ مفهوم byte addressing در این معماری وجود ندارد: آدرس 5 یعنی ششمین کلمه‌ی ۱۶ بیتی حافظه، نه بایت پنجم.

۲-۴ رجیستر فلگ و اجرای شرطی سراسری

ویژگی متمایزکننده‌ی PARS-16 این است: هر دستور، بدون استثنا، دو بیت شرط اجرا (CC) دارد. پیش از اجرای واقعی هر دستور، کنترلر بررسی می‌کند آیا شرط آن دستور با وضعیت فعلی رجیستر فلگ برقرار است یا نه. اگر برقرار نباشد، دستور عملاً به یک NOP تبدیل می‌شود: هیچ رجیستری نوشته نمی‌شود، هیچ حافظه‌ای تغییر نمی‌کند، فلگ‌ها دست‌نخورده می‌مانند و حتی پرش/جهش هم رخ نمی‌دهد (PC فقط

یک واحد افزایش پیدا می‌کند). این الگو که به آن اجرای پیش‌بینی شده^۱ می‌گویند دقیقاً همان مکانیزمی است که معماری ARM (تا نسخه‌ی ARMv7) روی تمام دستورات خود پیاده می‌کند [۲].

۲-۴-۱ رجیستر فلگ

چهار فلگ زیر در یک رجیستر ۴ بیتی نگهداری می‌شوند:

فلگ	نام	شرح
Z	Zero	یک می‌شود اگر نتیجه‌ی آخرین عملیاتِ فلگ‌گذار دقیقاً صفر باشد
N	Negative	برابر با بیت علامت (بیت ۱۵ام) نتیجه است
C	Carry	نقل رقم خروجی جمع‌کننده (برای SUB/ADD)
V	oVerflow	سرریز علامت‌دار (فرمول زیر)

فرمول سرریز برای ADD ($Rd = Rs1 + Rs2$):

$$V = (Rs1_{15} = Rs2_{15}) \text{ و } (Rd_{15} \neq Rs1_{15})$$

و برای SUB ($Rd = Rs1 - Rs2$):

$$V = (Rs1_{15} \neq Rs2_{15}) \text{ و } (Rd_{15} \neq Rs1_{15})$$

برای دستورهای منطقی/شیفت (SHR، SHL، XOR، OR، AND)، همیشه $C=0$ و $V=0$ تعریف می‌شود (فقط Z و N بر اساس نتیجه‌ی نهایی به‌روز می‌شوند).

¹ Predicated Execution

۲-۴-۲ بیت S: فعال سازی به روزرسانی فلگ

فقط هفت دستور فرمت R (بخش ۲-۵) یک بیت S در قالب خود دارند. اگر $S=1$ و شرط CC آن دستور برقرار باشد، رجیستر فلگ با نتایج تازه به روزرسانی می شود؛ اگر $S=0$ باشد (یا شرط برقرار نباشد)، فلگ ها دست نخورده می مانند. هیچ دستور دیگری (SYS, JR, JAL, B, LUI, MOVI, SW, LW, ADDI) اصلاً بیت S در قالب خود ندارد و به هیچ وجه فلگ عوض نمی کند.

۲-۴-۳ کد شرط دوبیتی (CC)

CC	نام	شرط اجرا	کاربرد معمول
00	AL (Always)	همیشه اجرا شود	پیش فرض بیشتر دستورها
01	EQ (Equal)	$Z=1$	برابری، پایان حلقه ی شمارشی
10	LT (Less Than)	$N=1$	مقایسه ی علامت دار (کوچک تر بودن)
11	VS (oVerflow Set)	$V=1$	تشخیص سرریز حسابی

چرا فقط این سه شرط

با ۲ بیت فقط ۴ حالت داریم که یکی از آن ها باید «همیشه» باشد (چون اکثر دستورهای یک برنامه ی معمولی اصلاً نمی خواهند شرطی باشند). این یعنی فقط ۳ «شرط واقعی» در دسترس است. ما EQ, LT و VS را انتخاب کردیم چون پرکاربردترین دسته های مقایسه (برابری، ترتیب، سرریز) را پوشش می دهند. شرط های معکوس مثل NE (نامساوی) یا GE (بزرگ تر مساوی) به طور مستقیم وجود ندارند — در بخش ۲-۴-۶ یاد می گیرید چطور آن ها را با ترکیب دو دستور شبیه سازی کنید. این محدودیت عمدی، دقیقاً همان مصالحه ای است که طراحان ISA واقعی هم با آن روبه رو می شوند: عرض کد شرط در برابر تعداد بیت های در دسترس برای بقیه ی دستور.

۴-۴-۲ مقایسه چگونه انجام می‌شود

هیچ دستور CMP مستقلی در جدول opcode وجود ندارد؛ به‌جایش از SUB با $Rd=R0$ و $S=1$ استفاده می‌شود (نتیجه‌ی تفریق دور ریخته می‌شود چون در $R0$ نوشتن بی‌اثر است، اما فلگ‌ها به‌روز می‌شوند):

```
CMP R1, R2 ; pseudo-op, equivalent to: SUB R0, R1, R2, S
```

۵-۴-۲ الگوی پرش شرطی: چرا دو سیکل کلاک

چون هیچ دستور پرشی اجازه‌ی فعال‌سازی فلگ (بیت S) ندارد، برای «پرش بر اساس یک مقایسه» همیشه به دو دستور پشت‌سرهم نیاز دارید: یکی برای محاسبه و ثبت فلگ، دیگری برای پرش شرطی که همان فلگ‌های تازه را می‌خواند:

```
CMP R1, R2 ; cycle 1: update flags based on R1-R2
B.EQ Loop ; cycle 2: if R1==R2, jump to Loop
```

۶-۴-۲ شبیه‌سازی شرط‌های معکوس (GE، NE، ...)

از آنجا که NE به‌طور مستقیم در دسترس نیست، از الگوی «رد شو / پرش بدون قید» استفاده کنید. برای مثال، برای پیاده‌سازی «اگر $R1$ با $R2$ برابر نبود به Target برو»:

```
CMP R1, R2
B.EQ Skip ; if equal, skip over the unconditional jump
B.AL Target ; otherwise (i.e. not equal), jump unconditionally
Skip:
```

این یک الگوی شناخته‌شده در معماری‌های اولیه‌ای است که مجموعه‌ی محدودی از کدهای شرط داشتند؛ تمرین آن به شما کمک می‌کند محدودیت‌های سخت‌افزاری واقعی را از نزدیک لمس کنید.

۵-۲ فرمت‌های دستور

فیلد CC (دو بیت، بیت‌های ۱۴-۱۵) و فیلد OPCODE (چهار بیت، بیت‌های ۱۰-۱۳) در تمام فرمت‌ها دقیقاً در همین موقعیت ثابت قرار دارند این یکنواختی، رمزگشایی سیگنال‌های کنترلی را به‌شدت ساده می‌کند.

MSB (bit 15)		LSB (bit 0)			
CC	OPCODE	Rd	Rs1	Rs2	S

فرمت R – عملیات ثبات به ثبات (۷ دستور، تنها فرمتی با بیت S)

MSB (bit 15)		LSB (bit 0)			
CC	OPCODE	Rd/Rs2	Rs1	imm4	

فرمت I – عملیات با مقدار فوری کوچک (بدون فلیگ)

MSB (bit 15)		LSB (bit 0)			
CC	OPCODE	Rd	imm7		

فرمت U – بارگذاری مقدار فوری بزرگ‌تر

MSB (bit 15)		LSB (bit 0)			
CC	OPCODE	offset10			

فرمت B/J – پرش نسبی (خود CC شرط پرش را می‌سازد؛ نیازی به فیلد رجیستر نیست)

MSB (bit 15)		LSB (bit 0)			
CC	OPCODE	Rs1	reserved		

فرمت JR – پرش غیرمستقیم به آدرس داخل یک رجیستر

دقت: جای فیلد رجیستر در JR با فرمت R فرق دارد

در فرمت R، فیلد Rs1 در بیت‌های 6..4 است. اما JR فرمت مستقل خودش را دارد و تنها فیلد رجیسترش در بیت‌های 9..7 می‌نشیند — یعنی همان‌جایی که در بقیه‌ی فرمت‌ها Rd قرار می‌گیرد. بیت‌های 6..0 رزرو و بی‌معنی‌اند.

پس در پیاده‌سازی، آدرسی که به پورت خواندن اول رجیستر فایل می‌دهید برای JR باید از بیت‌های 9..7 برداشته شود، نه 6..4. ساده‌ترین راه یک MUX سه‌بیتی روی آن آدرس است که با سیگنال کنترلی JR انتخاب می‌شود.

مثال: R1 JR برابر 0x3880 است (نه 0x3810).

MSB (bit 15)		LSB (bit 0)	
CC	OPCODE	func	reserved

فرمت SYS — دستورهای ویژه (HALT/NOP) فقط NOP

تغییر: دستور HALT حذف شد

کد عملیات 1111 از این پس فقط NOP است: هیچ رجیستری، هیچ خانه‌ای از حافظه و هیچ فلگی را عوض نمی‌کند و فقط PC را یکی جلو می‌برد. فیلد func دیگر معنایی ندارد و نادیده گرفته می‌شود.

پس برنامه‌چطور تمام می‌شود؟ با یک پرش بی‌قید به خودِ همان خانه: یک دستور B که برچسب مقصدش، برچسب خودِ همان خط است (آفست برابر منفی یک). اسمبلر همچنان HALT را می‌پذیرد و آن را به همین پرش ترجمه می‌کند، پس متن برنامه‌های شما تغییری نمی‌کند:

```
Stop: B Stop ; this is what HALT assembles to
```

چه چیزی برایتان ساده‌تر شد: سه بخش از مسیر داده حذف می‌شود — فلیپ‌فلاپ «متوقف‌شده»، منطق انجماد PC، و گیت‌هایی که هنگام توقف باید نوشتن در رجیستر فایل و حافظه را می‌بستند.

نکته‌ی مهم درباره‌ی فرمت I: در دستور SW، فیلدی که در LW/ADDI به‌عنوان «مقصد نوشتن» (Rd) خوانده می‌شود، در SW به‌عنوان «رجیستر منبع داده برای نوشتن در حافظه» (Rs2) تفسیر می‌شود —

موقعیت بیتی یکسان است، فقط معنایش فرق می‌کند. این دقیقاً همان ترفندی است که RISC-V هم در فرمت S خود استفاده می‌کند.

۶-۲ جدول کامل دستورات

عملکرد	S	فرمت	Mnemonic	Opcode
$Rd = Rs1 + Rs2$	بله	R	ADD	0000
$Rd = Rs1 - Rs2$ (پایه‌ی CMP)	بله	R	SUB	0001
$Rd = Rs1 \& Rs2$	بله	R	AND	0010
$Rd = Rs1 Rs2$	بله	R	OR	0011
$Rd = Rs1 \wedge Rs2$	بله	R	XOR	0100
شیفت منطقی چپ $Rs1$: مقدار شیفت $Rs2[3:0] =$	بله	R	SHL	0101
شیفت منطقی راست $Rs1$: مقدار $Rs2[3:0] =$ شیفت	بله	R	SHR	0110
$Rd = Rs1 + sext(imm4)$	خیر	I	ADDI	0111
$Rd = MEM[Rs1 + sext(imm4)]$	خیر	I	LW	1000
$MEM[Rs1 + sext(imm4)] = Rs2$	خیر	I	SW	1001
$Rd = sext(imm7)$ (بارگذاری مستقیم ثابت کوچک)	خیر	U	MOVI	1010
$Rd[8:0] = 0, Rd[15:9] = imm7$	خیر	U	LUI	1011
اگر CC برقرار: $PC = PC + 1 + sext(offset10)$	خیر	B	B	1100
اگر CC برقرار: $PC = PC + 1 + sext(offset10)$	خیر	J	JAL	1101
اگر CC برقرار: $PC = Rs1$ (بازگشت از زیرروال با $R7$ JR)	خیر	JR	JR	1110
	خیر	SYS	NOP	1111
:func=0001:NOP:func=0000				
HALT بدون عملیات؛ فقط $PC=PC+1$				

تمام آفست‌ها (imm4، imm7 در MOV، offset10) به صورت علامت‌دار دو موی (two's complement) تفسیر می‌شوند مگر آنکه خلاف آن گفته شود. آفست پرش نسبت به $PC+1$ (یعنی آدرس دستور بعدی) محاسبه می‌شود.

سه تعریف که با هشت‌بیتی شدن PC لازم شد

(۱) مقصد پرش پیمانه‌ی 256 حساب می‌شود. چون PC هشت بیت است:

$$PC_{\text{new}} = (PC + 1 + \text{sext}(\text{offset10})) \bmod 256$$

نتیجه‌ی مهم و شاید غیرمنتظره: کافی است هشت بیت پایین offset10 را بردارید و در یک جمع‌کننده‌ی هشت‌بیتی جمع کنید. این دقیقاً همان جواب را می‌دهد که بسط علامت به شانزده بیت و بعد برش. مثال واقعی: پرشی از آدرس 201 به آدرس 1، آفستش 201- است؛ هشت بیت پایانش 55 (مثبت!) می‌شود و $202 + 55 = 257 \equiv 1$ درست است. پس هر دو سبک پیاده‌سازی پذیرفته می‌شود.

(۲) JR فقط هشت بیت پایین Rs1 را برمی‌دارد، بیت‌های بالا نادیده گرفته می‌شوند. مثلاً اگر $Rs1 = 0x1207$ باشد، PC برابر 0x07 می‌شود.

(۳) JAL مقدار $PC+1$ را با صفر به شانزده بیت گسترش می‌دهد و در R7 می‌نویسد (نه بسط علامت - آدرس بی‌علامت است). پس اگر JAL در آدرس 0 باشد، R7 برابر 0x0001 می‌شود.

۷-۲ شبه‌دستورها

یک اسمبلر کمکی می‌تواند شبه‌دستورهای زیر را به دستورهای واقعی جدول بالا ترجمه کند:

شبه‌دستور	معادل واقعی	توضیح
CMP Rs1, Rs2	SUB R0, Rs1, Rs2, S	مقایسه بدون نوشتن نتیجه
MOV Rd, Rs	ADD Rd, Rs, R0	کپی مقدار بین رجیسترها
NOP	کد عملیات 1111	بدون عملیات
HALT	B SYS به خود خط	پرش به خود
RET	JR R7	بازگشت از زیرروال
LI Rd, #imm16	ADDI+LUI	بخش ۱-۷-۲

۱-۷-۲ محدودیت LI و ساختن ثابت‌های بزرگ

افزوده‌ی جدید

Rd, imm7 LUI مقدار imm7 را در بیت‌های ۱۵..۹ می‌گذارد و بیت‌های ۸..۰ را صفر می‌کند؛ یعنی به‌تنهایی فقط مضارب ۵۱۲ را می‌سازد. ADDI هم فقط ۸..۷- جابه‌جا می‌کند. پس LI نمی‌تواند هر عدد ۱۶ بیتی دلخواهی را بسازد — ولی برای اعداد حدی که برای آزمون سرریز لازم دارید کاملاً کافی است:

مقدار	معنی	دستورها
0x8000	منفی‌ترین عدد	LUI R1, 0x40
0x7FFF	مثبت‌ترین عدد	LUI R1, 0x40 سپس ADDI R1, R1, -1

دقت کنید 0x40 است نه 0x80: فیلد imm7 هفت بیت است و به اندازه‌ی نه بیت شیفت می‌خورد، پس $0x8000 = 9 \ll 0x40$. نوشتن 0x80 خطای «خارج از بازه» می‌دهد. نگارش علامت‌دارش 64- است و اسمبلر هر دو را می‌پذیرد. بهتر است اصلاً درگیر این حساب نشوید و LI Rd, value بنویسید؛ اسمبلر خودش تصمیم

می‌گیرد `MOVI` کافی است یا `LUI` یا `ADDI+LUI`، و آدرس برجسب‌های بعدی را هم درست حساب می‌کند.

۸-۲ نحو زبان اسمبلی

- هر خط: یک برجسب اختیاری (`Label:`)، یک دستور یا شبه‌دستور، و یک توضیح اختیاری بعد از؛.
- کد شرط به‌صورت پسوند با نقطه می‌آید: `B.EQ`، `B.LT`، `B.VS`؛ نبود پسوند یعنی `AL` (بدون قید).
- فعال‌سازی فلگ با پسوند `S`، در انتهای دستورهای فرمت `R`: مثلاً `ADD R1, R2, R3, S`.
- اعداد: دهمی معمولی (12)، مبنای شانزده با پیشوند `0x` (`0x1F`)، دودویی با پیشوند `0b`.
- نشانی‌دهی حافظه در `SW/LW` به‌سبک `offset(base)`: مثلاً `LW R1, 4(R2)` یعنی $R1 = \text{MEM}[R2+4]$ ، و `SW R3, -2(R2)` یعنی $\text{MEM}[R2-2] = R3$.

۱-۸-۲ مثال کامل: جمع اعداد 1 تا N

```

MOVI R1, 10      ; R1 = N = 10
MOVI R2, 0        ; R2 = sum = 0
MOVI R3, 1        ; R3 = i = 1

Loop:  CMP  R3, R1
      B.LT Body    ; if i < N, continue (see note below)
      B.EQ Body    ; the i == N case must also run once more
      B.AL Done

Body:  ADD  R2, R2, R3 ; sum += i
      ADDI R3, R3, 1  ; i++
      B.AL Loop

Done:  SW   R2, 0(R0) ; store the result in MEM[0]
```

HALT

این مثال به خوبی نشان می‌دهد که نبودِ شرط LE (کوچک‌تر یا مساوی) مستقیم، برنامه‌نویسی را کمی پرحرف‌تر می‌کند دقیقاً همان بده‌بستانی که در بخش ۲-۴-۶ توضیح داده شد.

فصل ۳

راهنمای معماری میکرو و قرارداد رابط اجباری

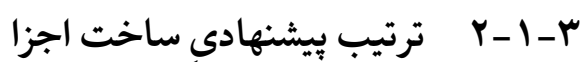
۱-۳ مسیر داده‌ی تک‌سیکل

برای بازه‌ی سه‌هفته‌ای، مسیر داده‌ی تک‌سیکل^۱ الزام پایه است: هر دستور دقیقاً در یک پالس کلاک، از واکنشی تا نوشتن نتیجه، کامل اجرا می‌شود.

۱-۱-۳ نمای کلی بلوکی

شکل زیر ارتباط اجزای اصلی را نشان می‌دهد (جهت سیگنال‌های کنترلی با خط‌چین قرمز مشخص شده است). این یک نقشه‌ی مفهومی است، نه سیم‌کشی دقیق جزئیات هر باس و هر مالتی‌پلکسر را خودتان طراحی می‌کنید.

¹ Single-Cycle



۱۰. مالتی پلکسرهای عملوند/نتیجه/آدرس بعدی + گیت کردن سیگنال‌های نوشتن با cond_met

¹ Sign-Extend

۱۱. پورت‌های دیباگ (بخش ۳-۳) - در پایان؛ یک MUX روی آدرس پورت خواندن دوم و یک پورت خواندن روی حافظه‌ی داده

۲-۳ جدول سیگنال‌های کنترلی

این جدول طرح مرجع پیشنهادی است (می‌توانید نام سیگنال‌ها را عوض کنید، اما رفتار باید معادل باشد). ردیف‌ها بر اساس گروه دستور خلاصه شده‌اند چون دستورهای هم‌فرمت، کنترل یکسانی دارند (فقط عملکرد ALU در فرمت R فرق می‌کند که مستقیماً برابر با سه بیت پایین OP CODE است).

گروه	RegW	RegDst	MemRd	MemWr	ALUSrcA	ALUSrcB	ALUFunc	WBSel	PCSrc
(0000-0110) SHR..ADD	1	Rd	0	0	Rs1	Rs2	=OPCODE[2:0]	ALU	seq
(0111) ADDI	1	Rd	0	0	Rs1	imm4	ADD	ALU	seq
(1000) LW	1	Rd	1	0	Rs1	imm4	ADD	Mem	seq
(1001) SW	0	-	0	1	Rs1	imm4	ADD	-	seq
(1010) MOVI	1	Rd	0	0	Zero	imm7	ADD	ALU	seq
(1011) LUI	1	Rd	0	0	-	-	-	LUI	seq
(1100) B	0	-	0	0	-	-	-	-	Branch
(1101) JAL	1	R7	0	0	-	-	-	PC+1	JAL
(1110) JR	0	-	0	0	-	-	-	-	JR
(1111) NOP	0	-	0	0	-	-	-	-	seq

راهنمای ستون‌ها: RegW: نوشتن روی رجیستر فایل مجاز است؟ RegDst: آدرس مقصد نوشتن از کجا می‌آید (فیلد Rd، یا همیشه R7 برای JAL)؟ ALUSrcB/ALUSrcA: ورودی‌های ALU از کجا می‌آیند؟ WBSel: مقداری که در رجیستر فایل نوشته می‌شود از کدام منبع است (خروجی ALU، داده‌ی حافظه، نتیجه‌ی الحاق LUI، یا PC+1 برای JAL)؟ PC Src: مقدار بعدی PC (توالی عادی PC+1، هدف پرش نسبی B، هدف JAL، یا محتوای Rs1 برای JR)؟

نکته‌ی حیاتی: گیت‌کردن با شرط

هر مقدار «۱» در ستون‌های RegW، MemWr و هر انتخاب غیر از «توالی عادی» در PCSrc، باید علاوه‌بر جدول بالا، با بیت cond_met هم AND شود. یعنی:

$$\text{RegW_final} = \text{RegW_table} \text{ AND } \text{cond_met}$$

و مشابه برای MemWr_final و انتخاب JR/JAL/Branch در PCSrc (اگر شرط برقرار نبود، PCSrc عملاً به «توالی عادی» برمی‌گردد، حتی اگر دستور از نوع JR/JAL/B بوده باشد). به‌همین ترتیب: $\text{FlagWrite_final} = \text{S_bit}$ AND FlagCapable AND cond_met (که FlagCapable یعنی این دستور از گروه فرمت R است ستون آخر جدول قبل).

۳-۲-۱ پیشنهاد برای ساخت خود واحد کنترل

ساده‌ترین و تمیزترین روش: یک قطعه‌ی ROM کوچک که با OPCODE (۴ بیت) آدرس‌دهی می‌شود و محتویات هر سطر آن، دقیقاً همان سیگنال‌های کنترلی جدول بالا برای آن opcode است (الگوی کلاسیک «control store»). این را می‌توانید با ادیتور داده‌ی ROM در Digital به‌سادگی پر کنید. سپس یک لایه‌ی ترکیبی کوچک، خروجی این ROM را با AND cond_met می‌کند تا سیگنال‌های نهایی به‌دست بیایند.

۳-۳ قرارداد پین الزام سخت برای نمره‌دهی خودکار**این بخش را با دقت بخوانید**

اگر مدار نهایی شما (NN_CPU.dig) دقیقاً همین پین‌ها را با همین نام و همین عرض نداشته باشد، هیچ‌کدام از تست‌های خودکار روی آن اجرا نخواهند شد و نمره‌ی آن بخش صفر ثبت می‌شود (نه به‌خاطر اشتباه در طراحی، بلکه چون ابزار تست اصلاً نمی‌تواند پین‌ها را پیدا کند). نام‌ها به بزرگی/کوچکی حروف حساس‌اند.

پین	جهت	عرض	توضیح
CLK	ورودی	1	کلاک سیستم
RESET	ورودی	1	ریست همگام، فعال در 1: باید PC و فلگ‌ها را صفر کند و رجیسترها — رجیستر فایل ریست نمی‌شود
DBG_RSEL	ورودی	3	شماره‌ی رجیستری که می‌خواهیم از پورت دیباگ ببینیم
DBG_EN	ورودی	1	۱ = حالت مشاهده (آدرس پورت خواندن دوم ← DBG_RSEL)
DBG_MADDR	ورودی	16	آدرس مشاهده در حافظه‌ی داده
PC_OUT	خروجی	8	مقدار جاری PC
IR_OUT	خروجی	16	دستور در حال واکنشی/اجرا (خروجی خام حافظه‌ی دستور)
FLAGS_OUT	خروجی	4	ترتیب بیت‌ها: {V, C, N, Z} (V=بیت ۳ ... Z=بیت ۰)
DBG_ROUT	خروجی	16	مقدار رجیستر DBG_RSEL (پورت سوم) (پورت دوم)
DBG_MOUT	خروجی	16	محتوای DMEM[DBG_MADDR]
HALT_OUT	خروجی	±	پس از HALT یک می‌شود حذف شد

پورت‌های دیباگ هیچ اثری روی درستی معماری ندارند و صرفاً پنجره‌ی مشاهده برای تست خودکارند، دقیقاً مثل پورت دیباگ/JTAG. در پردازنده‌های واقعی، پیاده‌سازی داخلی شما (نام سیم‌ها، تعداد فایل، ساختار سلسله‌مراتبی) کاملاً آزاد است؛ فقط این پین‌ها در بالادستی‌ترین فایل باید دقیقاً

همین‌طور باشند.

الزامی - تنظیم قطعات حافظه (این بخش در نسخه‌ی قبلی جا افتاده بود)

علاوه بر نام پین‌ها، دو قطعه‌ی حافظه هم باید دقیقاً این‌طور تنظیم شوند، وگرنه هیچ تستی روی مدار شما اجرا نمی‌شود:

قطعه	Label	Program Memory	Data/Addr Bits
حافظه‌ی دستور (ROM)	IMEM	تیک‌خورده	16 / 8
حافظه‌ی داده (RAM)	DMEM	بدون تیک	16 / 16
هر ROM دیگر (مثلاً جدول کنترل)	هر نامی جز این دو	بدون تیک	آزاد

چرا: فایل تست مرجع، برنامه را با دستور program(...) از بیرون در حافظه‌ی دستور شما می‌ریزد و داده‌ی اولیه را با `memory DMEM(addr)=value;` در حافظه‌ی داده می‌گذارد. Digital برای این کار باید دقیقاً یک قطعه پیدا کند؛ اگر پیدا نکند یا دو تا پیدا کند، خطا می‌دهد و کل آن فایل تست رد می‌شود.

خطاهایی که خواهید دید:

- تیک Program Memory نخورده باشد: "No program memory found! The program

memory needs to be flagged as such."

- دو ROM تیک‌خورده باشد (مثلاً جدول کنترل را هم تیک زده باشید):

"multipleRomsFound"

- برچسب RAM چیزی جز DMEM باشد: "No memory DMEM found in the model!"

تیک Program Memory را در پنجره‌ی تنظیمات همان قطعه‌ی ROM پیدا می‌کنید، و برچسب را در فیلد Label.

DBG_EN دقیقاً چه کاری می‌کند

DBG_EN «حالت مشاهده» را روشن می‌کند. حداقل کاری که باید بکند این است که آدرس پورت خواندن دوم رجیستر فایل را به DBG_RSEL سوئیچ کند. اما اگر بخواهید پنجره‌ی مشاهده‌ی حافظه (DBG_MOUT/DBG_MADDR) را هم با همین سیگنال گیت کنید، مجاز است. هر دو پیاده‌سازی قبول می‌شوند، چون تست‌های مرجع پیش از خواندن هر پنجره‌ی دیباگ، همیشه DBG_EN را یک می‌کنند. اگر پورت حافظه‌تان همیشه فعال است هم مشکلی نیست؛ فقط توجه کنید که این پورت هرگز نباید بنویسد، فقط می‌خواند.

RESET چه چیزی را صفر می‌کند و چه چیزی را نه

در نسخه‌ی ۲۴ تیر نوشته شده بود «PC، رجیسترها و فلگ‌ها». این عبارت ابهام داشت و باعث پرسش شد، چون قطعه‌ی آماده‌ی RegisterFile در Digital اصلاً پایه‌ی ریست ندارد. تعریف درست: RESET فقط رجیسترهای وضعیت را صفر می‌کند، یعنی PC و رجیستر فلگ. رجیستر فایل ریست نمی‌شود و لازم هم نیست بشود. این دقیقاً رفتار پردازنده‌های واقعی است: در ARM، RISC-V و x86 هم محتوای رجیسترهای عمومی بعد از ریست تعریف نشده است و فقط PC و رجیسترهای وضعیت مقدار مشخص دارند. مسئولیت مقداردهی اولیه با برنامه است — به همین دلیل هر برنامه‌ی نمونه‌ی این پروژه با چند MOVI شروع می‌شود. تست‌های مرجع هم بر همین اساس نوشته شده‌اند و هرگز فرض نمی‌کنند رجیستری در آغاز مقدار خاصی دارد.

دو نکته‌ی عملی که آزمودیم

(۱) پین ورودی که به هیچ‌جا وصل نباشد خطا نمی‌دهد؛ مدارتان عادی اجرا می‌شود (فقط تست مربوط به آن رد می‌شود). اما پین خروجی که هیچ سیگنالی به آن وارد نمی‌شود — یعنی سیمی از خروجی هیچ قطعه‌ای به آن وصل نیست — خطای “No output connected to a wire” می‌دهد و مدار اصلاً اجرا نمی‌شود، چون مقدار آن سیم نامعین است. پس هر پنج پین خروجی حتماً باید

از داخل مدار سیگنال بگیرند.

۲) برای CLK هم یک پین ورودی ساده و هم قطعه‌ی Clock کار می‌کند و تست‌کیس‌ها با هر دو اجرا می‌شوند؛ انتخابش با شماست.

۳-۱ نکته‌ی پیاده‌سازی: پورت خواندن سوم رجیستر فایل رجیستر فایل: دو

پورت خواندن، نه سه

اکثر رجیستر فایل‌هایی که در آموزش می‌سازید، دو پورت خواندن دارند (برای Rs1 و Rs2). برای پورت دیباگ، یک پورت خواندن سوم و کاملاً مستقل لازم دارید که آدرسش از DBG_RSEL می‌آید. ساده‌ترین راه: از همان آرایه‌ی رجیستری، یک خروجی multiplexer دیگر هم بگیرید.

الزام پورت سوم حذف شد

دلیلش این بود که قطعه‌ی آماده‌ی RegisterFile کتابخانه‌ی Digital فقط دو پورت خواندن دارد (Da و Db) و راهی برای افزودن پورت سوم به آن وجود ندارد؛ پس الزام قبلی با توصیه به استفاده از همان قطعه در تضاد بود.

راه جدید – فقط یک مالتی‌پلکسر: آدرس پورت خواندن دوم را از یک MUX بگیرید:

آدرسی که به پورت خواندن دوم می‌رود	DBG_EN
0 (اجرای عادی) Rs2 (یا Rd در دستور SW)	
1 (حالت مشاهده) DBG_RSEL	

و DBG_ROUT همان خروجی پورت دوم است. تست‌ها DBG_EN را فقط بعد از پایان برنامه و با 0CLK = فعال می‌کنند، پس هیچ نوشتنی رخ نمی‌دهد و اجرای برنامه مختل نمی‌شود.

اگر از قطعه‌ی آماده استفاده می‌کنید یک نکته‌ی مستقل هم هست: آن قطعه R0 را صفر نگه نمی‌دارد (آزمودیم: با $Rw=0$ مقدار نوشته می‌شود و با $Ra=0$ پس می‌آید). پس باید بیرونش: we را با شرط « $0 \neq Wa$ » گیت کنید و خروجی هر پورت را از یک MUX رد کنید که در آدرس صفر، صفر بدهد.

اگر با سازوکار تست خودکار آشنا نیستید، پیش از هر چیز نمونه‌ی کوچک پوشه‌ی `examples/hello_test` را اجرا کنید؛ در چند دقیقه می‌بینید که یک تست چطور نوشته و اجرا می‌شود.

۳-۴ سؤالات متداول طراحی

آیا استفاده از قطعه‌ی آماده‌ی Arithmetic یا Multiplexer کتابخانه‌ی Digital مجاز است؟ بله. تمرکز این پروژه بر معماری و سازمان‌دهی سطح بالا است، نه طراحی گیتی ALU. استفاده از قطعات آماده‌ی کتابخانه (که خودشان مدار دیجیتال هستند، نه کد نوشته‌شده) کاملاً مجاز است.

آیا می‌توانم مدار را به صورت چندفایلی (سلسله‌مراتبی) بسازم؟ بله، و توصیه هم می‌شود (رجیستر فایل، ALU و کنترلر را در فایل‌های جدا بسازید). فقط یادتان باشد طبق بخش ۱-۵، همه‌ی فایل‌ها باید در تحویل نهایی همراه هم باشند.

اگر بخواهم عرض حافظه را عوض کنم چه می‌شود؟ نکنید. عرض 8 bit برای PC و آدرس حافظه آدرس حافظه‌ی دستور بخشی از قرارداد اجباری است (بخش ۳-۳). آدرس حافظه‌ی داده هم 16 bit است و همان قدر اجباری.

Bibliography

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2013.
- [2] ARM Limited, *ARM Architecture Reference Manual, ARMv7-A and ARMv7-R Edition*, 2014, conditional (predicated) instruction execution, Chapter A8.
- [3] H. Neemann, “Digital: A digital logic designer and circuit simulator,” <https://github.com/hneemann/Digital>, 2026, accessed July 2026.
- [4] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, 2024.